PeerBoard Writeup

Max Moir

May 5, 2026

1 Intro

For this project, I made a node for a decentralized peer-to-peer bulletin-board-style chat. I implemented a React, Typescript, and TailwindCSS frontend that communicates with to a Rust libp2p backend using a websocket. This allows the user's node to connect with others, subscribe to different topics, and share messages. I also started to implement a Rendevous protocol for matching with other nodes, which can find other nodes searching for challengers.

2 Design Decisions

The main design decisions I was proud of in this project were related to the frontend UI. I was happy with the design decisions relating to UI implementation and which frameworks to use. These led to an aesthetically pleasing frontend UI that works smoothly for the user. For example, dynamically filtering the messages based on topic works well, and the controls work seamlessly together. An alternative I considered was controlling topic sections from a sidebar, similar to Slack or Discord channels. I tested this format, but I thought it changed the experience from a bulletin board to more of a messaging server. Overall, I think the approach I settled on is better and is more intuitive for the user. The theme toggling button also works well, although in hindsight it wasn't a priority.

In terms of the backend, I think I made multiple poor design decisions early on that hindered my progress in the later stages of the project. If I were to do it again, one design decision I would change would be unifying the message data type. In my implementation, there is not one single source of truth for what a message should contain, with different data types for the database, swarm, frontend, and bridge api. In addition to this, I decided to first interface between the frontend and the server, then between the server and the swarm runner. This is honestly a pointless separation, and it made it much harder to develop, because I had to redefine commands in three or more places to pass a message from the UI to the database.

3 One Surprise

One aspect of the Rust libp2p API that surprised me was the amount of flexibility and configurability for different options and protocol types. The different protocols, both used in this project and present in the examples for libp2p, were extensive. This project also demonstrated how these protocols interact with and complement each other, in a way I hadn't considered previously. Another thing that took me a long time to figure out was the proper initialisation of the swarm object. Connecting to the bootstrap node was especially finicky. Even when I thought I had implemented it correctly, it later silently broke for the rendezvous protocol.

4 Grade Estimation

For this Assignment, I estimate that I will get a grade in the high B to low A range. This is because I have not implemented the requirements for a high A range grade, but I have exceeded the requirements for the lower grade bands in other ways. For example, the B band requirements are all fulfilled, including message persistence, deduplication, and presence after restart. The interface also covers all bulletin board topics, such as subscribing, unsubscribing, posting, and viewing the feed for a specific

topic. In addition to all of this, it has a well put-together frontend UI that dynamically updates without user interaction. The user can change their name, send messages under different topics, and view the exact time each message was posted. I also started to implement some parts of the A-band requirements, such as the rendezvous protocol. This works for registering nodes and discovering challengers, although a challenge cannot be sent. Hence, the requirements for the B and C-bands are met, with additional effort placed on making an aesthetically pleasing and reactive UI.